Lean In Circle Recommendation Engine

What the app does

FastAPI reads members, Circles, topics, and activity from SQLite. It removes unsuitable Circles, scores the rest, adds variety, and explains each suggestion. The same data produces the same results.

The website (`/static/index.html`) lets nontechnical teammates search members and browse readable recommendations. The top-right **Developer API** button opens `/docs` to try `/recommendations/{user_id}` and inspect JSON responses and scores. Browser network tools show response times.

How recommendations work

The app excludes already-joined, full, unlisted, archived, and inactive Circles. A Circle needs activity within 60 days, plus at least one post or five chat messages in the supplied 30-day window.

Scores combine interests (60%), profile fit (10%), and community health (30%). I chose these weights; the assignment did not provide them. Interests come first, activity matters next, and profile fit plays a smaller role because profiles can be incomplete. These are starting values that I would adjust using real member feedback. They consider industry, meeting format, posts, and chat volume. Missing locations are not guessed. Members without interests receive active, open communities; more complete leader profiles give a small boost. Similar topics receive a penalty, with at most two Circles per main topic. Every result has a factual explanation.

Why limit 10 results as default for OpenAPI? Ten provides a manageable shortlist and a smaller response. Clients can request 1–50. All eligible Circles are still scored. The website shows six results at a time.

Choices and tradeoffs

For a short assignment, simple scoring rules are easier to set up, debug, and explain than models needing training data or extra services. Calculating scores on each request avoids maintaining saved recommendation lists and reflects saved capacity changes. Activity totals do not update automatically; the demo uses September 21, 2026 to keep results repeatable.

Supporting more users

- Move to PostgreSQL. Find about 200 suitable Circles using indexed filters and tested pgvector or Elasticsearch search, then rank them. Measure speed rather than assume retrieval takes under 10 milliseconds.
- Reuse onboarding suggestions and scoring data through Redis. Use Kafka/SQS background workers to update activity and memberships.
- Add sign-in, automated checks, monitoring, and seat checks that prevent simultaneous joins from exceeding capacity.
- Measure accepted joins and continued participation. Compare scoring changes through A/B tests and check which communities receive exposure.